THE UNIVERSITY OF AZAD JAMMU AND KASHMIR

DIGITAL IMAGE PROCESSING LAB

LAB TASK # 02

Digital Images, Pixels, Color, and Basic Transformations

Student Name	**Fatima Arshad**
Roll Number	**2024-SE-24**
Lab Instructor	**Engr. Fahad Nisar**
Semester	**Fifth**
Session	**2024–28 (SE)**
Working Environment	**Anaconda / Jupyter Notebook**

Department of Software Engineering

1. Objective

The objective of this lab is to use Python, NumPy, OpenCV, and Matplotlib to load and inspect a digital image, understand pixel coordinates and color channels, convert between color representations, crop and resize an image, create a grayscale histogram, and compare different bit depths.

2. Working Environment and Required Libraries

The practical work was carried out using Anaconda and Jupyter Notebook with a Python 3 (ipykernel) environment. The required libraries were imported and tested successfully.

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

print("All libraries imported successfully!")
```

Observed import check: All libraries imported successfully!

```
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt

print("NumPy:", np.__version__)
print("OpenCV:", cv.__version__)
print("Matplotlib:", plt.matplotlib.__version__)
print("All libraries imported successfully!")
```

```
NumPy: 2.4.6
OpenCV: 5.0.0
Matplotlib: 3.11.0
All libraries imported successfully!
```

3. Loading and Inspecting the Image

OpenCV loads a color image in BGR channel order. The image array shape is reported as (height, width, channels), and uint8 indicates that each channel value is stored as an 8-bit unsigned integer.

```
image = cv.imread("my_image.jpg")
if image is None:
    raise FileNotFoundError("Image file not found. Check the path.")

print("Image shape:", image.shape)
print("Data type:", image.dtype)
```

```python
image_path = r"C:\Users\pc\Desktop\dip-lab-02\my_image.jpg"

image_bgr = cv.imread(image_path)

if image_bgr is None:
    raise FileNotFoundError("Image nahi mili. File ka naam check karo.")

print("Image loaded successfully!")
print("Image shape:", image_bgr.shape)
print("Data type:", image_bgr.dtype)
```

```
Image loaded successfully!
Image shape: (352, 236, 3)
Data type: uint8
```

Note: this recorded image is 352 pixels high and 236 pixels wide. The handout requests an image of at least 300 × 300 pixels, so check the original image dimensions and use a qualifying image if the instructor applies that requirement strictly.

4. Pixel Values and Intensity Range

A pixel is accessed using [row, column] indexing. The first coordinate is the row (y position) and the second is the column (x position). For a color image, each pixel contains three channel values.

```
top_left_pixel = image[0, 0]
center_pixel = image[height // 2, width // 2]

print("Top-left pixel (BGR):", top_left_pixel)
print("Center pixel (BGR):", center_pixel)
print("Global minimum:", image.min())
print("Global maximum:", image.max())
print("Per-channel minimum (B, G, R):", image.min(axis=(0, 1)))
print("Per-channel maximum (B, G, R):", image.max(axis=(0, 1)))
```

The minimum and maximum values describe the intensity range present in the image. For an 8-bit image, valid channel values range from 0 to 255.

```python
# Image ki height, width aur channels
height, width, channels = image_bgr.shape

print("Height:", height)
print("Width:", width)
print("Number of channels:", channels)

# Top-left pixel
print("Top-left pixel (BGR):", image_bgr[0, 0])

# Center pixel
print(
    "Center pixel (BGR):",
    image_bgr[height // 2, width // 2]
)

# Global minimum aur maximum
print("Global minimum:", image_bgr.min())
print("Global maximum:", image_bgr.max())

# Har channel ka minimum aur maximum
print("Channel minimums (B, G, R):", image_bgr.min(axis=(0, 1)))
print("Channel maximums (B, G, R):", image_bgr.max(axis=(0, 1)))
```

```
Height: 352
Width: 236
Number of channels: 3
Top-left pixel (BGR): [174 186 190]
Center pixel (BGR): [116 140 168]
Global minimum: 0
Global maximum: 255
Channel minimums (B, G, R): [0 0 8]
Channel maximums (B, G, R): [235 246 255]
```

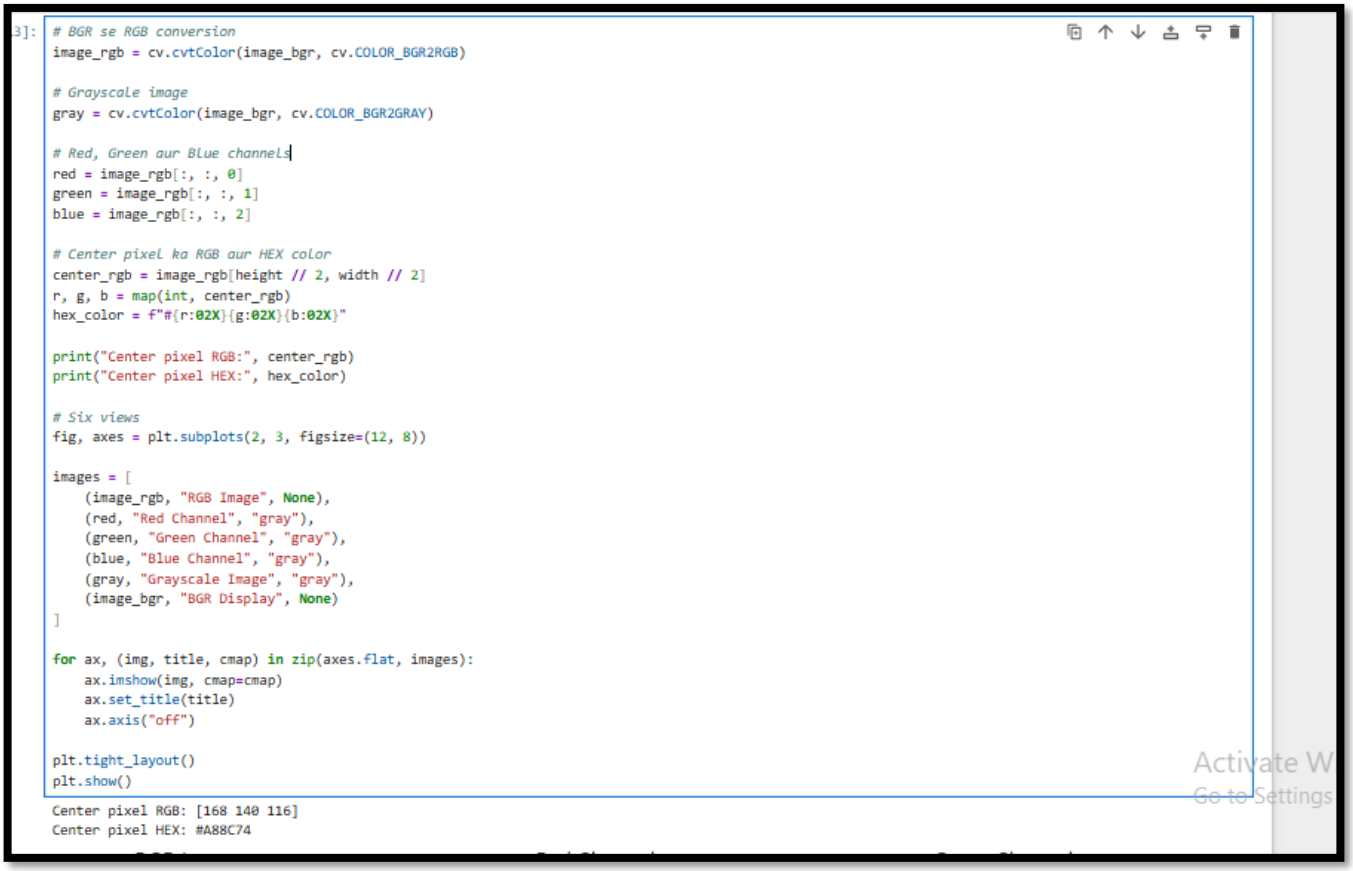
5. Color Conversion and Channel Analysis

OpenCV stores color images in BGR order, while Matplotlib expects RGB order for normal color display. Converting BGR to RGB prevents the red and blue channels from appearing swapped. A grayscale image represents intensity without separate color information.

```
rgb = cv.cvtColor(image, cv.COLOR_BGR2RGB)
gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
blue, green, red = cv.split(image)

center_rgb = rgb[height // 2, width // 2]
hex_color = "#{:02X}{:02X}{:02X}".format(*center_rgb)
print("Center pixel RGB:", center_rgb)
print("Center pixel HEX:", hex_color)
```

The six-panel figure should display the RGB image, red channel, green channel, blue channel, grayscale image, and the original unconverted BGR array displayed directly with Matplotlib. Use titles and hide axes. Individual channel images should use a gray colormap.



```
3]: # BGR se RGB conversion
image_rgb = cv.cvtColor(image_bgr, cv.COLOR_BGR2RGB)

# Grayscale image
gray = cv.cvtColor(image_bgr, cv.COLOR_BGR2GRAY)

# Red, Green aur Blue channels
red = image_rgb[:, :, 0]
green = image_rgb[:, :, 1]
blue = image_rgb[:, :, 2]

# Center pixel ka RGB aur HEX color
center_rgb = image_rgb[height // 2, width // 2]
r, g, b = map(int, center_rgb)
hex_color = f"#{r:02X}{g:02X}{b:02X}"

print("Center pixel RGB:", center_rgb)
print("Center pixel HEX:", hex_color)

# Six views
fig, axes = plt.subplots(2, 3, figsize=(12, 8))

images = [
    (image_rgb, "RGB Image", None),
    (red, "Red Channel", "gray"),
    (green, "Green Channel", "gray"),
    (blue, "Blue Channel", "gray"),
    (gray, "Grayscale Image", "gray"),
    (image_bgr, "BGR Display", None)
]

for ax, (img, title, cmap) in zip(axes.flat, images):
    ax.imshow(img, cmap=cmap)
    ax.set_title(title)
    ax.axis("off")

plt.tight_layout()
plt.show()

Center pixel RGB: [168 140 116]
Center pixel HEX: #A88C74
```

Activate W
Go to Settings



6. Cropping, Resizing, and Saving

Cropping selects a region of the image using array slicing. The crop is 100 × 100 pixels. It is then resized to 200 × 200 pixels. Resizing changes the dimensions of the selected region; it does not restore details that were not present in the crop.

```
crop = image[y_start:y_start + 100, x_start:x_start + 100]
  print("Crop shape:", crop.shape)

  resized = cv.resize(crop, (200, 200))
  rgb_resized = cv.cvtColor(resized, cv.COLOR_BGR2RGB)

  saved = cv.imwrite("resized_crop.png", resized)
  print("Image saved successfully:", saved)
```

Use crop coordinates that keep the full 100 × 100 region inside the image. The completed run reported 'Image saved successfully: True'.

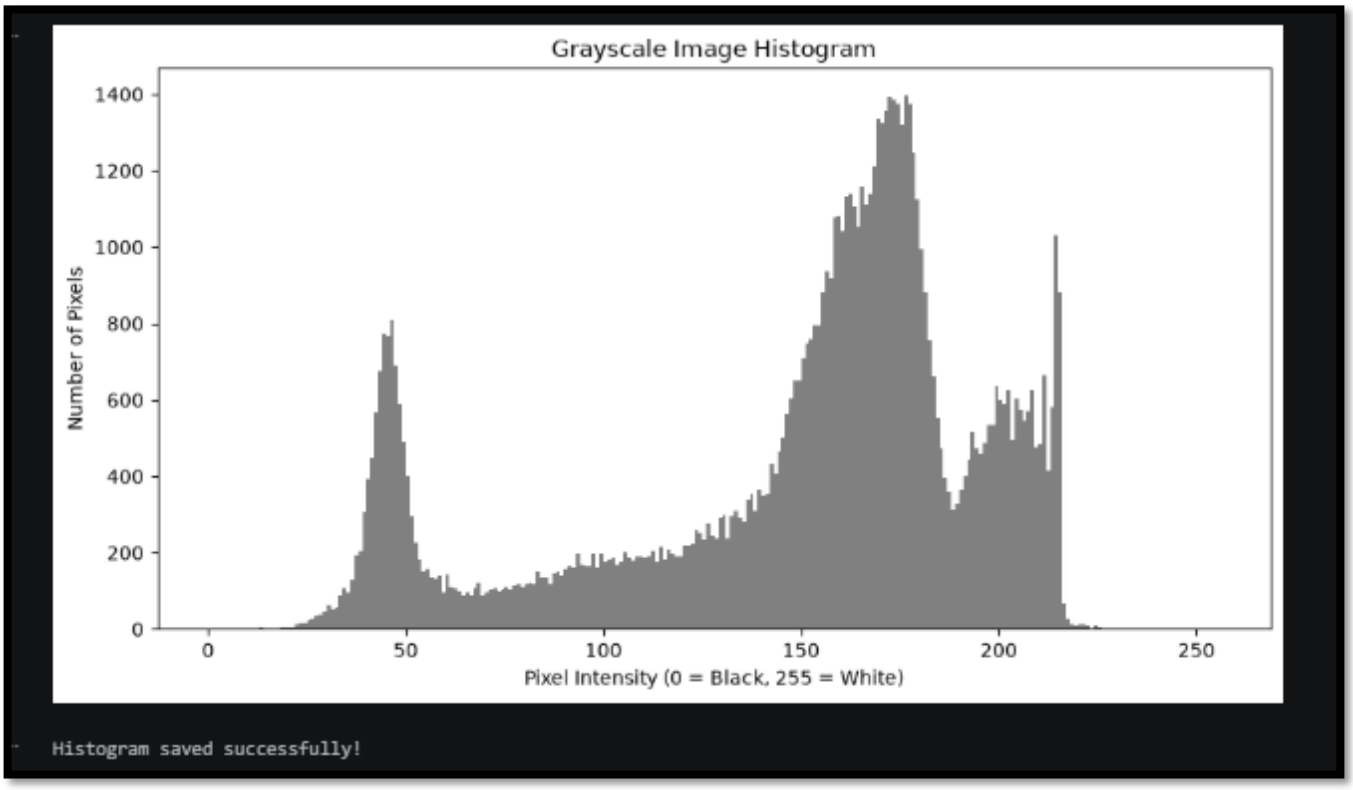


7. Grayscale Histogram

A grayscale histogram counts how many pixels fall at each intensity level from 0 to 255. It can show whether pixel intensities are concentrated in darker, middle, or brighter ranges. A histogram does not show where those pixels are located in the image and does not describe the image's full spatial structure.

```
gray_values = gray.ravel()
plt.hist(gray_values, bins=256, range=(0, 256))
plt.title("Grayscale Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Number of Pixels")
plt.tight_layout()
plt.savefig("grayscale_histogram.png")
plt.show()
```

Interpretation: After inserting the histogram screenshot, describe the distribution visible in your own plot. For example, say that the image is mainly dark if most counts are concentrated toward lower intensities, mainly bright if they cluster toward higher intensities, or more balanced if they are spread across a broad range. Do not select a description unless it matches your actual histogram.



8. Bit-Depth Demonstration

The original 8-bit grayscale gradient can represent 256 intensity levels. Quantizing it to fewer bits reduces the number of available intensity levels, so the gradient looks less smooth and more visibly stepped.

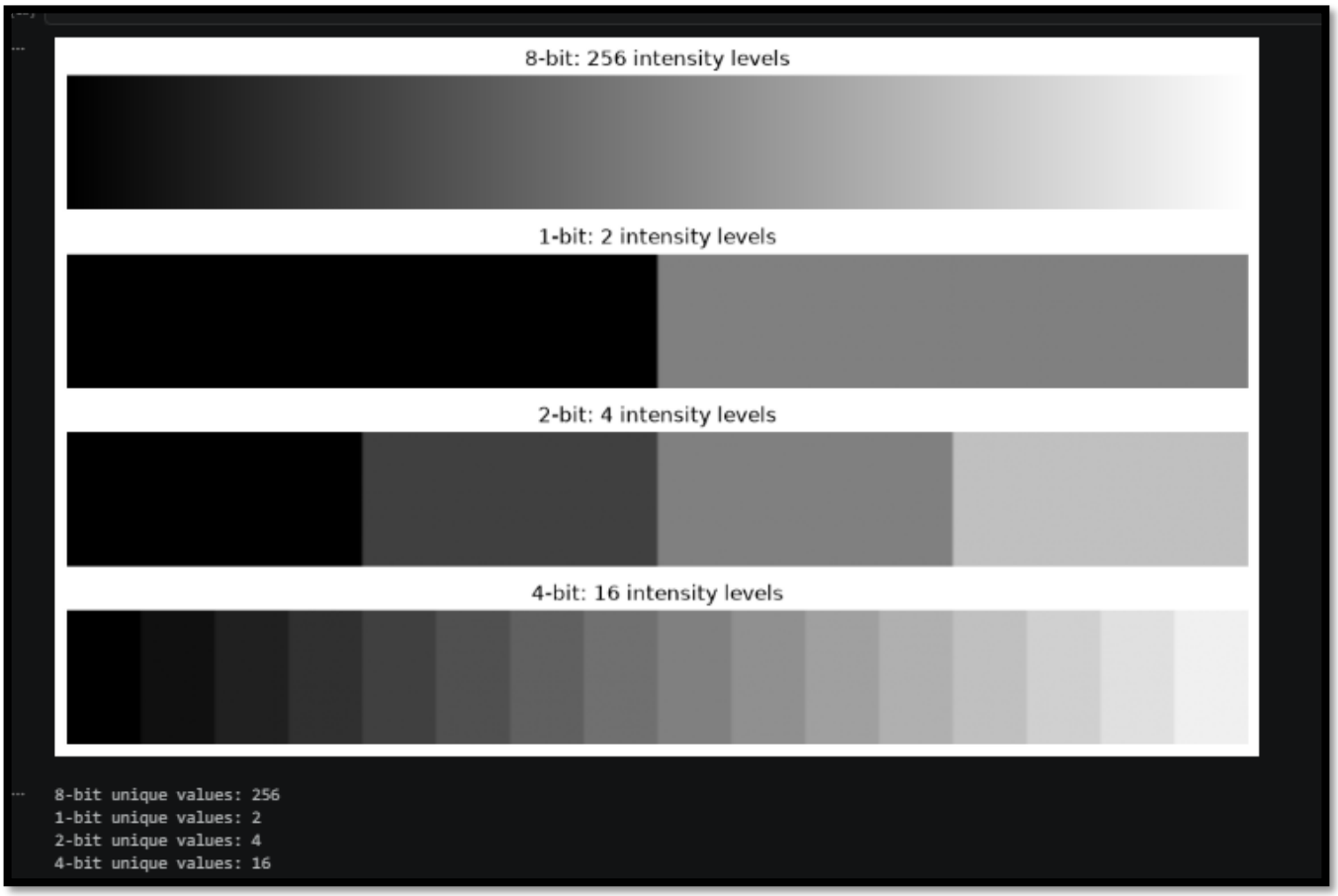
```
gradient = np.tile(np.arange(256, dtype=np.uint8), (50, 1))

gradient_8bit = gradient
gradient_1bit = (gradient // 128) * 255
gradient_2bit = np.round(gradient / 85) * 85
gradient_4bit = np.round(gradient / 17) * 17

print("8-bit unique values:", len(np.unique(gradient_8bit)))
print("1-bit unique values:", len(np.unique(gradient_1bit)))
print("2-bit unique values:", len(np.unique(gradient_2bit)))
print("4-bit unique values:", len(np.unique(gradient_4bit)))
```

Recorded unique-value counts:

Resolution refers to the number of pixels used to represent an image (its dimensions). Bit depth refers to the number of bits used to represent each pixel's intensity or color channel. Increasing resolution can represent more spatial detail; increasing bit depth allows more intensity levels.



9. Output Files and Final Verification

The project should contain the Python script, the original input image, the saved resized crop, the saved grayscale histogram, the .gitignore file, and this report. The virtual-environment folder (.venv) should not be included in the submission. Since this work was performed in Jupyter Notebook, include evidence from the environment actually used and confirm with the instructor if a VS Code-only environment is required.

```
# .gitignore file content
.venv/
```

10. Conclusion

In this lab, I practised loading and inspecting a digital image with OpenCV and NumPy, reading pixel values, examining intensity ranges, converting BGR to RGB and grayscale, separating color channels, and generating a HEX color value. I also cropped and resized an image, saved output files, plotted a grayscale histogram, and compared 8-bit intensity levels with 1-bit, 2-bit, and 4-bit quantization. These activities helped me understand how image dimensions, color representation, pixel intensity, and bit depth affect digital image processing.

****